SHELL SCRIPTING BEST PRACTICES

Before writing any code, know exactly what it is you want your script to do and plan for the unexpected (error-handling, false user-input, etc.).
Write clean code, use indents and use comments to explain parts of your code!!!
Scripting works better if you feel comfortable using shell-commands on the host-system.
Use functions.
Use variables.
Open another terminal and try out parts of the code to see its output on screen (good for those extra spaces that screw up IF-statements).
In case of the above, sed is your friend.
More friends: awk, head, tail, wc, grep, ps, etc...
Use Good Indentation
It really makes code far more readable and, thus, more maintainable. This is especially true when you have more than three levels of logic. Indentation makes it easy to see the basic form of your script's logic. It doesn't matter much whether how many space you indent, though most people seem to use 4 spaces or 8. Just make sure that your do's and done's line up and you'll be fine.

```
#!/bin/bash

if [ $# -ge 1 ] && [ -d $1 ]; then
    for file in `ls $1`
    do
        if [ $debug == "on" ]; then
            echo working on $file
        fi
        wc -l $1/$file
    done
else
    echo "USAGE: $0 directory"
    exit 1
fi
```

- **Provide usage statements**

Usage statements can help anyone running your scripts -- even yourself two years later -- to know what they're expected to provide as

arguments.

```
if [ $# == 0 ]; then
    echo "Usage: $0 filename"
    exit 1
fi
```

Use Sensible Commenting

SHELL SCRIPTING BEST PRACTICES

Provide comments that explain your code, especially when it's complicated, but don't explain the obvious lines, but explain every command that you're using or the important ones get lost in the mix.

```
username=$1

# make sure the account exists on the system
grep ^$username: /etc/passwd
if [ $? != 0 ]; then
    echo "No such user: $username"
    exit 1
fi
```

- **Exit with a return code when something goes wrong**

Even if you don't think you'll look at them, returning a non-zero return code when something goes wrong is a good idea. Someday, you might want an easy way to check what went wrong in the script and a return code of 1 or 4 or 11 might help you figure this out very quickly.

```
echo -n "In what year were you born?> "
read year

if [ $year -gt `date +%Y` ]; then
    echo "Sorry, but that's just not possible."
    exit 2
fi
```

Use functions rather than repeating groups of commands
Functions can also make your code readable and more maintainable. Don't bother if it's only one command you're using repeatedly, but if it's easy to separate a handful of focused commands, it's worth the trouble. If you have to make a change later on, you will only have to make it on one place.

```
function lower()
{
    local str="$@"
    local output
    output=$(tr '[A-Z]' '[a-z]'<<<"${str}")
    echo $output
}
```

- **Give your variables meaningful names**

Unix admins generally bend over backwards to avoid typing a few extra characters, but don't do this in your scripts. Take the extra time to give your variables meaningful names and to use some consistency in your naming.

SHELL SCRIPTING BEST PRACTICES

```
#!/bin/bash

if [ $# != 1 ]; then
    echo "Usage: $0 address"
    exit 1
else
    ip=$1
fi
```

- **Check that arguments are of the correct type**

You can save yourself a lot of trouble if you check to make sure the arguments provided to your script are of the type expected before you start to use them. Here's an easy way to check if an argument is numeric.

```
if ! [ "$1" -eq "$1" 2> /dev/null ]
then
  echo "ERROR: $1 is not a number!"
  exit 1
fi
```

- **Check if needed files actually exist**

It's easy to check that a file exists before trying to use it. Here's a simple check to see whether the first argument is actually a file on the system.

```
if [ ! -f $1 ]; then
    echo "$1 -- no such file"
fi
```

- **Send output to /dev/null**

Sending command output to /dev/null and telling users what went wrong in a more "friendly" way can make your scripts easier on those who need to run them.

```
if [ $1 == "help" ]; then
    echo "Sorry -- No help available for $0"
else
    CMD=`which $1 >/dev/null 2>&1`
    if [ $? != 0 ]; then
        echo "$1: No such command -- maybe misspelled or not on your search
path"
        exit 2
    else
        cmd=`basename $1`
        whatis $cmd
```

SHELL SCRIPTING BEST PRACTICES

```
    fi
fi
```

- **Make use of error codes**

You can use return codes within your script to determine if a command got the expected result or not.

```
# check if the person is still logged in or has running processes
ps -U $username 2> /dev/null
if [ $? == 0 ]; then
    echo "processes:" >> /home/oldaccts/$username
    ps -U $username >> /home/oldaccts/$username
fi
```

- **Give feedback**

Don't forget to tell the people running your scripts what they need to know. They shouldn't have to read your code to be reminded where you created a file for them -- especially if it's not in the current directory.

```
...
date >> /tmp/report$$
echo "Your report is /tmp/report$$"
```

- **quote all parameter expansions**

If you're using characters that expand within your scripts, don't forget to use quotes so that you don't get a very different result than you expected.

```
#!/bin/bash

msg="Be careful to name your files *.txt"
# this will expand *.txt
echo $msg
# this will not
echo "$msg"
```

- **Use $@ when referring to all arguments**

The `$@` variable lists all the arguments provided to your script and it's easy to use as we see in this little script excerpt.

SHELL SCRIPTING BEST PRACTICES

```
#!/bin/bash

for i in "$@"
do
    echo "$i"
done
```